Engineering a Lazy Decision Procedure for LIA* in cvc5: Strategies, Algorithms, and an Experimental Account

Claude (Anthropic)

June 2026

Abstract

We describe a sequence of engineering and algorithmic improvements to the lazy decision procedure for the *additive closure* (star) operator over linear integer arithmetic implemented in cvc5, following the approximation-based approach of Levatich et al. (VMCAI 2020). Starting from an implementation that timed out on the hardest instances of a BAPA/MAPA-derived benchmark family, we develop and evaluate seven strategies: subsolver refinement under push/pop, main-solver cell enumeration, propositional and semantic cell generalization, constant-size reduction lemmas via partial sums, decomposition-guided cell discovery with a decoupled endgame and decision-strategy witness feedback for satisfiable instances, and counterexample-guided synthesis of over-approximating homogeneous cuts for unsatisfiable instances. Each step was driven by a measured failure of the previous one; we report the negative results alongside the positive ones. The final configuration solves all 480 benchmarks within a 100-second timeout (median well under one second), including instances on which every reference solver in our comparison set timed out, with no answer discrepancies.

1 Problem and Background

1.1 The *star-contains* operator

A literal

$$\ell \;=\; \mathit{int.star\text{-}contains}\big(\lambda x_1 \ldots x_n.\; p(\mathbf{x}),\; v_1, \ldots, v_n\big)$$

asserts that the vector $\mathbf{v} = (v_1, \ldots, v_n)$ of integer terms belongs to the *star* (additive closure) of the set

$$S \;=\; \{\, \mathbf{x} \in \mathbb{Z}^n \mid p(\mathbf{x}) \wedge \mathbf{x} \geq 0 \,\}, \qquad S^* \;=\; \Big\{\; \textstyle\sum_{j=1}^{k} \mathbf{s}_j \;\Big|\; k \geq 0,\; \mathbf{s}_j \in S \;\Big\},$$

where the empty sum yields the zero vector. The sets S^* are exactly the non-negative semilinear sets; the operator arises when encoding BAPA (Boolean algebra with Presburger arithmetic) and multiset (MAPA) constraints, where each summand abstracts one Venn-region cardinality pattern.

1.2 The reduction of Levatich et al.

The decision procedure reduces ℓ to plain linear integer arithmetic:

1. Normalize p into DNF, $p \Leftrightarrow D_1 \vee \cdots \vee D_m$; each disjunct (a *cell*) is a convex polyhedron, so $S = S_1 \cup \cdots \cup S_m$.

2. The closure distributes over union as a Minkowski sum: $S^* = S_1^* + \cdots + S_m^*$, since the summands of any sum can be grouped by the cell they came from.

3. For each cell, Normaliz computes *module generators* g and a *Hilbert basis* $\{h_k\}$ of its recession cone, so its integer points are $\{\, g + \sum_k l_k h_k \mid l_k \geq 0 \,\}$. The semigroup S_j^* is captured by a multiplier $\mu_g \geq 0$ per generator (how many copies are taken) and $l_k \geq 0$ per ray, coupled by $\mu_g = 0 \Rightarrow l_k = 0$.

4. Summing the contributions over all cells gives the *star constraints*: a linear system over fresh multipliers whose satisfiability is equivalent to $\mathbf{v} \in S^*$. The procedure emits the reduction lemma $\ell \Leftrightarrow \mathit{star}$.

The *eager* strategy builds the whole DNF up front; since the DNF explodes (the benchmark predicates contain up to 23 if-then-else terms), the *lazy* strategy instead discovers one cell per refinement round, from a model of a region not yet covered, and asserts the equivalence only *tentatively*: while cells are missing, star_k under-approximates S^*, so the lemma $g_k \Rightarrow (\ell \Leftrightarrow \mathit{star}_k)$ is guarded by a fresh Boolean decision variable g_k biased to true, and the previous guard is deactivated ($\neg g_{k-1}$) each round. Once every cell is covered, the exact, unguarded equivalence is emitted; this is what allows refuting ℓ.

1.3 Architecture in cvc5

The procedure lives in an extension of the arithmetic theory (`LiaStarExtension`), invoked at full effort with the linear solver's candidate model. Each round it (i) emits a non-negativity lemma and a split on $p[\mathbf{v}]$ (if $\mathbf{v}$ itself satisfies p, it is a single-summand member); (ii) checks whether the candidate model already satisfies $p[\mathbf{v}]$ or the last under-approximation (the *model-value shortcut*); and otherwise (iii) runs one refinement round. Cells are discovered from models of a persistent incremental *subsolver* seeded with $p \wedge \mathbf{y} \geq 0$ (over fresh Skolem constants $\mathbf{y}$ substituted for the lambda's bound variables) and refined with the negations of the cells found so far.

2 Strategies for the refinement loop

2.1 Accumulate vs. push/pop (options `arith-liastar-push-pop`)

The baseline (*accumulate*) asserts the negation of each newly discovered cell once, monotonically; incremental mode allows repeated `checkSat` calls. The *push/pop* variant exploits the observation that each round's refined formula $\neg(D_1 \vee \cdots \vee D_k)$ *subsumes* the previous round's: it pops the previous refined formula and asserts the single cumulative one in a fresh frame, so the subsolver always holds exactly two assertions and the per-assertion caches stay bounded.

Measured: on the hard instance the two are indistinguishable (both time out around 10^3 rounds and 3.2 GB); on the full set accumulate is slightly ahead (463 vs. 460 of 480 solved at 100 s), because popping discards learned clauses and re-clausifies the cumulative formula (quadratic total work). Memory was dominated by the main solver, not the subsolver, so the push/pop advantage never materialized. *Lesson:* the suspected bottleneck (subsolver assertion accumulation) was wrong; measure before optimizing.

2.2 Main-solver enumeration (option `arith-liastar-main-solver`)

This variant removes the subsolver entirely: the lambda's variables become fresh Skolems $\mathbf{y}$ *in the main solver*, constrained through a chain of *stage guards* ($h_0 \Rightarrow \mathit{base}$, and per cell D_k a fresh h_k with $h_k \Rightarrow h_{k-1}$ and $h_k \Rightarrow \neg D_k$), and the next cell is read off the main solver's own candidate

Algorithm 1: Boolean cell generalization (`generalizeCell`)

Input: formula *base*, atoms $a_1..a_m$ with model values $b_1..b_m$
Output: keep flags $k_1..k_m$

$\sigma \leftarrow \{a_i \mapsto b_i\}$; **if** $\text{eval}_3(base, \sigma) \neq true$ **then return** all-keep;
for $i \leftarrow 1$ **to** m **do**
  $\sigma[a_i] \leftarrow unknown$;
  if $\text{eval}_3(base, \sigma) = true$ **then** $k_i \leftarrow false$ **else** $\sigma[a_i] \leftarrow b_i$;;
  $k_i \leftarrow true$;

model. A naïve completeness test (wait until the SAT solver fixes $\neg h$ at level 0) livelocks; the working mechanism is a per-stage *driver lemma*

$$\ell \;\Rightarrow\; \big(p[\mathbf{v}] \vee star_k \vee h_k\big),$$

which is satisfiability-preserving (any model with $\mathbf{v} \in S^*$ outside the current approximation extends to one placing $\mathbf{y}$ in an uncovered cell; the empty-sum member $\mathbf{v} = 0$ must be included in the star branch) and forces an asserted-but-uncertified literal to either certify itself or exhibit a fresh cell. An adversarial review found and we fixed two soundness bugs: the missing empty-sum branch (wrong `unsat` on predicates with empty S), and enumeration lemmas not being re-sent after user-context pops (wrong `sat` in incremental mode), fixed by re-queueing all lemmas through the user-context-dependent lemma cache each round.

Measured: sound (zero discrepancies) but dominated by the subsolver architecture (447 vs. 463 solved; 2.5× slower on the common set): every refinement round pays for the entire input search, and at most one cell is harvested per full-effort check.

2.3 Cell generalization (options `arith-liastar-generalize`, `-generalize-semantic`)

The cell read from a model fixes the truth value of *every* atom of the predicate, making cells maximally fine: the enumeration walks the arrangement of all atoms rather than the predicate's coarse cell structure. *Boolean generalization* computes a prime implicant of the propositional skeleton: greedily drop literals while a three-valued evaluation of *base* under the partial assignment still yields *true* (Algorithm 1). Soundness requires only that the shrunken conjunction still implies *base*; dropping conjuncts keeps the cell convex, the model point stays inside (so progress is preserved), and overlapping cells are harmless because the Minkowski-sum argument needs only $S = \bigcup_j S_j$.

Measured: a clear win and now the default — cells shrink to about half their literals, 466/480 solved (3 newly solved, none lost), and 4.4× faster on the commonly solved set.

Semantic generalization strengthens the test to arithmetic entailment: literal l may be dropped iff $(D \setminus \{l\}) \wedge \neg base$ is unsatisfiable. The implementation is core-guided: one `checkSat` with all cell literals as *assumptions* against a persistent probe subsolver holding $\neg base$; the unsat-assumptions core drops everything outside it at once, then a greedy deletion pass (equalities first) minimizes the core. *Measured:* mechanically effective (cells halve again) but a net loss (465 solved, 2.25× slower): probe latency plus fatter cells (larger Hilbert bases, more multipliers per cone) outweigh the extra merging. Kept optional, default off.

2.4 Partial sums (option `arith-liastar-partial-sums`)

Each round's reduction lemma contains sum constraints spanning *all* cones found so far, so cumulative lemma content grows quadratically in the round count. Defining running partial-sum

Skolems $P_k[i] = P_{k-1}[i] + \mathit{contribution}_k[i]$ as unconditional lemmas (sound: they constrain only fresh Skolems, satisfiable by zeros) collapses the guarded lemma to the constant-size $\mathbf{v} = P_k$. *Measured:* a negative result — memory got *worse* (4.4 vs. 3.3 GB on the hard instance) and 13 fewer instances solved. Two mechanisms: the per-cone definitional volume (tens of lemmas per round) dominates with a large constant, and the all-equality star forces dis-equality splitting when negated (*ARITH_SPLIT_DEQ* jumped $14 \to 663$), where the fat star offered cheap escapes (violate one multiplier bound). The quadratic-lemma-content hypothesis for the memory wall was thereby refuted. Kept for reference, default off.

3 Solving satisfiable hard instances

The hardest family (`card/fol_0000099--0000106`) resisted every strategy above. Diagnosis on `fol_0000100` established three facts. First, the instance is satisfiable but $A \wedge p[\mathbf{v}]$ is not (verified standalone): the vector genuinely needs ≥ 2 summands. Second, cell discovery was *not* the bottleneck once guided (below): all five summand bit-patterns needed by a hand decomposition appear within ∼30 rounds. Third, the failure was the *endgame*: concluding `sat` requires the main solver to find integer multipliers decomposing $\mathbf{v}$ over the discovered cones — an ILP over hundreds of Skolems — and it never gets to finish, because (i) the extension's check runs *before* integer branching, so fractional candidate models fail the shortcut and trigger yet another refinement, growing the ILP under the solver's feet, and (ii) the star's fresh equality atoms receive default SAT phases, so the guard is *propagated* false before ever being decided — measured: the guard was never true across 643 rounds.

Three mechanisms together solve this class (`fol_0000100`: timeout $\to$ 0.7 s):

Guided enumeration (option `arith-liastar-guided`, default on). Every summand of a non-negative decomposition satisfies $\mathbf{y} \leq \mathbf{v}$ componentwise. The subsolver query is therefore first checked under the assumptions $\{y_i \leq \hat{v}_i\}$, where $\hat{\mathbf{v}}$ is the candidate model's value of $\mathbf{v}$, with an unbiased fallback when the biased query is unsatisfiable — so completeness (the unbiased `unsat` signalling full coverage) is untouched. In particular, coordinates with $\hat{v}_i = 0$ force $y_i = 0$, pruning the cell search to the subspace relevant for decomposing $\mathbf{v}$.

Decoupled endgame (option `arith-liastar-endgame`= N, default 10). Every N newly discovered cones, a *fresh* subsolver solves $F_0 \wedge \mathit{star}_k$, where F_0 are the arithmetic trail facts *fixed at decision level 0* (`Valuation::isFixed`) — using unfixed trail facts poisons the query with branch decisions (in one measured run, the trail had pinned $n = 1$ inside a region already known contradictory). The fresh solver runs branch-and-bound to completion, unencumbered by stale guards, frozen phases, and lemma churn. With guidance, it answers `sat` at ∼30 cones and yields a concrete multiplier witness.

Witness feedback by decision strategy. A guarded hint lemma $d \Rightarrow (x_1 = c_1 \wedge \cdots)$ with d phase-preferred true *never* engages: with hundreds of pinned variables, some are always already bound differently on the trail, so $\neg d$ is propagated round after round (measured: the hint guard was false at every single round). The working mechanism is a `cvc5` *decision strategy* (`LiaStarHintStrategy`): while a hint is active, the SAT solver is handed the guard and then each pinned equality as its next decisions, building the witness assignment directly; one hint is active per literal (competing witnesses pin the same Skolems to different values). The hint is

sound unconditionally — the guard is fresh — and once the witness assignment is complete, the model-value shortcut accepts it.

4 Refuting unsatisfiable hard instances: over-approximating cuts

The four remaining timeouts (`fol_0000098/0000107`, both encodings) are *unsatisfiable*: the lazy procedure could refute them only by completing the enumeration, which is hopeless (the eager DNF also times out). They need the *over-approximation half* of the VMCAI approach, which had not been implemented. Our instance of it:

Proposition 1 (Homogeneous cuts)**.** *If* $\mathbf{c} \cdot \mathbf{y} \geq 0$ *holds for every* $\mathbf{y} \in S$*, then* $\mathbf{c} \cdot \mathbf{v} \geq 0$ *holds for every* $\mathbf{v} \in S^*$*.*

Proof. S^* consists of finite sums of members of S (the empty sum being 0); non-negativity of a linear form is preserved under addition. □

Such *cuts* can therefore be asserted for $\mathbf{v}$ unconditionally. Crucially, cuts may also be *conditional on the zero-forced restriction*: if v_i is the constant 0 (which happens frequently — input equalities like $u_{13} = 0$ are substituted into the literal), then every summand has $y_i = 0$, so validity needs to hold only on $S \cap \{y_i = 0\}$ — a strictly larger space of valid cuts. The MAPA instance `fol_0000107` is refuted only by such a conditional cut ($4\,u_{10} - u_{14} - u_{17} - u_{20} - u_{23} - u_{26} \geq 0$, valid only where the comparison bits are pinned to zero).

Cuts are synthesized by counterexample-guided search (Algorithm 2), invoked whenever the endgame finds $F_0 \wedge \mathit{star}_k$ unsatisfiable. Three details matter in practice:

- *Sparsity.* The coefficient search is constrained by an escalating L_1 budget $\sum_i |c_i| \leq K$, $K \in \{4, 9, 16, 30\}$. An unconstrained box admits arbitrary dense candidates that the validity oracle rejects one at a time; the cuts that exist in practice are sparse, and the budget makes the CEGIS loop converge (measured offline: 8 cuts, ∼33 iterations in total, refute `fol_0000107`).
- *Sample soundness.* Module generators of discovered cones are genuine points of S; those with zeros on the restricted coordinates lie in the restricted region and soundly constrain the search ($\mathbf{c} \cdot p \geq 0$). Hilbert-basis *rays* must *not* be used: a ray of an unrestricted cell has zeros on its pinned coordinates yet describes a direction outside the restriction, and requiring non-negativity on it excludes exactly the valid conditional cuts (this was a measured implementation bug; the counterexample loop handles unbounded directions soundly instead).
- *Termination of the outer loop.* Once the accumulated cuts make F_0 unsatisfiable, nothing more is needed: the cuts are ordinary lemmas, so the main solver derives `unsat` by itself — no special reporting mechanism, hence no new soundness obligations.

Measured: all four remaining instances are refuted in 0.2–2.0 seconds (the conditional-cut and L_1 refinements were each necessary for the MAPA variant), with no effect on the rest of the suite.

5 Soundness summary

Every mechanism emits only standard theory lemmas; the procedure's answers are justified as follows. `sat` is concluded only when the candidate model satisfies $p[\mathbf{v}]$ (single summand) or star_k (an under-approximation of S^*, witnessing membership); hints and guards are fresh Booleans, so the guarded lemmas are satisfiability-preserving, and guided/endgame components only *steer* the

Algorithm 2: Conditional cut synthesis (`synthesizeCuts`, per endgame-unsat)

Input: literal ℓ with vector $\mathbf{v}$; fixed facts F_0; cuts C; samples P (restricted cone generators); zero-forced coordinates Z

for *round* $\leftarrow 1$ **to** 8 **do**
  if $F_0 \wedge C$ unsat **then return** (main solver will conclude unsat);
  $\hat{\mathbf{v}} \leftarrow$ model of $F_0 \wedge C$;
  foreach $K \in \{4, 9, 16, 30\}$ **do**
    for *iter* $\leftarrow 1$ **to** 60 **do**
      find $\mathbf{c}$ with $\sum_i |c_i| \le K$,;
        $\mathbf{c} \cdot p \ge 0 \ \forall p \in P, \quad \mathbf{c} \cdot \hat{\mathbf{v}} \le -1$;
      if *none* **then break** (next K);
      check validity: $\mathit{base} \wedge \bigwedge_{i \in Z} y_i{=}0 \wedge \mathbf{c} \cdot \mathbf{y} \le -1$;
      if *unsat* **then** emit lemma $\mathbf{c} \cdot \mathbf{v} \ge 0$; add to C; **continue outer else** add the model point to P;
  if *no cut found* **then** record failure (give up after two consecutive, reset on new cones);
    break;

search. unsat is concluded by the main solver from (i) the exact equivalence once enumeration completes, or (ii) cut lemmas, each individually valid for S^* (Proposition 1, restricted variant), conflicting with the input. Generalization preserves $D \Rightarrow \mathit{base}$ (propositionally or semantically), keeps the model point, and tolerates overlap; the subsolver's unbiased unsat remains the completeness certificate under guidance because the bias is applied only as assumptions with a fallback.

6 Experimental results

All experiments: production build of cvc5 with Normaliz, 480 benchmarks (`sls-reachability`, arith/card × BAPA/MAPA encodings), 100 s timeout, 12-way parallel harness, `OMP_NUM_THREADS=1`, 6 GB memory cap. No answer discrepancy was observed in any configuration, neither among our configurations nor against the four reference solvers recorded in the comparison data (SLS-reachability variants, an eager LIA encoding, and two SQLSolver variants).

Table 1: Solved instances per configuration (480 benchmarks, 100 s).

configuration	sat	unsat	timeout	solved
old recorded lazy baseline	272	178	30	450
lazy accumulate	283	180	17	463
lazy push/pop	280	180	20	460
lazy main-solver enumeration	273	174	33	447
lazy + boolean generalization	285	181	14	466
lazy + semantic generalization	284	181	15	465
lazy + partial sums	272	178	30	450
guided + endgame (no cuts)	296	180	4	476
final default (+ cuts)	**296**	**184**	**0**	**480**

The final default configuration — boolean generalization, guided enumeration, decoupled endgame

with decision-strategy witness feedback, and conditional homogeneous cuts — solves the entire suite. Previously impossible instances now answer in under two seconds, including several MAPA instances on which the reference SLS solver itself timed out.

7 Options added

option	default	role
`arith-liastar-generalize`	on	boolean cell generalization
`arith-liastar-guided`	on	decomposition-guided cell discovery
`arith-liastar-endgame=N`	10	decoupled endgame every N cones
`arith-liastar-cuts`	on	conditional homogeneous cut synthesis
`arith-liastar-generalize-semantic`	off	semantic generalization
`arith-liastar-push-pop`	off	cumulative subsolver refinement
`arith-liastar-main-solver`	off	subsolver-free enumeration
`arith-liastar-partial-sums`	off	constant-size reduction lemmas
`arith-liastar-patient`	off	refinement throttling experiment

8 Lessons

Three methodological points carried the project. *Measure before optimizing*: the first two memory-motivated strategies (push/pop, partial sums) targeted a bottleneck that profiling later refuted. *Localize by experiment*: the satisfiable-instance breakthrough came from tracing that the needed cones were found early and that the guard was never true — each fix was forced by a measured failure of the previous mechanism, down to the level of SAT-solver phase propagation. *Negative results are load-bearing*: semantic generalization and partial sums lost on this suite but their machinery (the probe oracle, the validity-by-assumptions pattern) became essential components of the cut synthesis that closed the final instances.